



High Performance Computing

Dr.Bheemappa H
IIIT, Sricity
bheemappa.h@iiit.in

M2 : Measures of Parallel Algorithms

54

- Analytical Modeling of Parallel Programs:
 - Sources of Overhead in Parallel Programs
 - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Parallel Platforms: Models (SIMD, MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

M2 : Measures of Parallel Algorithms

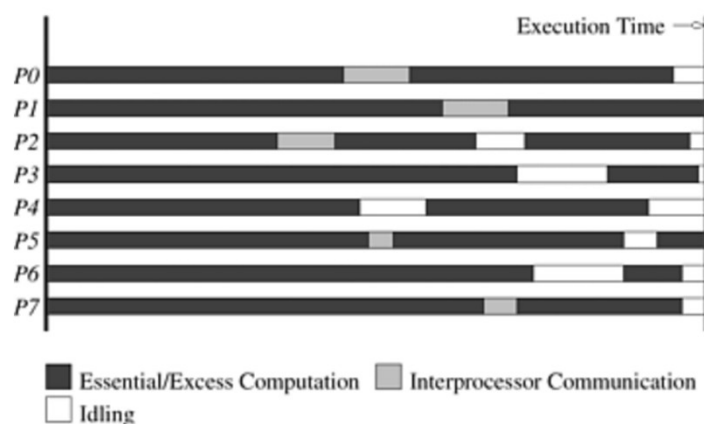
55

- Analytical Modeling of Parallel Programs:
 - Sources of Overhead in Parallel Programs
 - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Parallel Platforms: Models (SIMD, MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

Sources of Overhead in Parallel Programs

56

- If I use two processors, shouldn't my program run twice as fast?
 - **No** - a number of overheads, including wasted computation, communication, idling, and contention cause degradation in performance.



total time collectively spent : $T_{all} = pT_P$

$$T_o = pT_P - T_S.$$

- Performance Metrics for Parallel Systems: Execution Time :
 - time that elapses from the moment the first processor starts to the moment the last processor finishes execution.

M2 : Measures of Parallel Algorithms

57

- Analytical Modeling of Parallel Programs:
 - Sources of Overhead in Parallel Programs
 - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Parallel Platforms: Models (SIMD,MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

Five Performance Metrics for Parallel Systems

58

- Execution Time
- Overhead
- Speedup
- Efficiency
- Cost.

Measures of Parallel Algorithms

59

- **Execution Time**

- **Sequential algorithm** : Evaluated in terms of its execution time, expressed as a function of the size of its input.
- **Parallel Algorithm**: Input size but also on the number of processing elements used, and their **relative computation and interposes communication speeds**.
- **An algorithm must therefore be analyzed in the context of the underlying platform.**

```
for i = 1 to n do  
  C[i] ← A[i] × B[i]
```

- **Interprocess Interaction**

- to interact and communicate data (e.g., intermediate results)

- **Idling** :

- Due to many reasons such as load imbalance, synchronization, and presence of serial components in a program

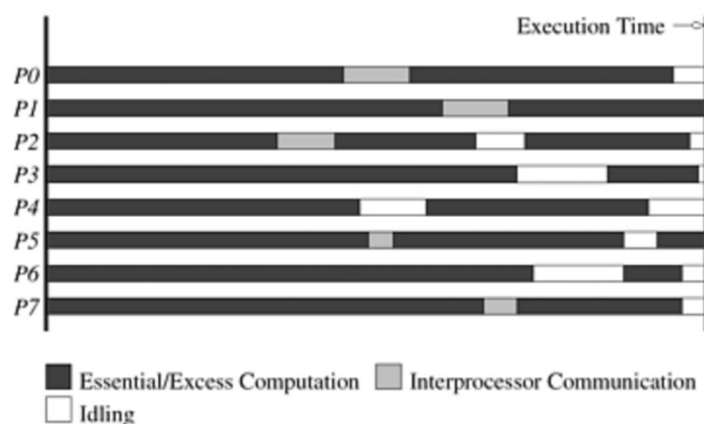
- **Excess Computation** :

- some computations are performed multiple times on different processing elements.

A parallel system is a combination of a parallel algorithm and an underlying platform.

Sources of Overhead in Parallel Programs

- a number of overheads, including wasted computation, communication, idling, and contention cause degradation in performance.



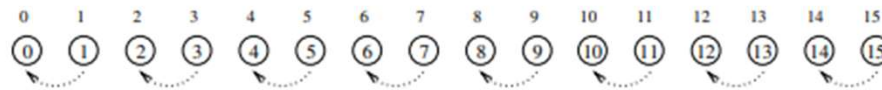
total time collectively spent : $T_{all} = pT_P$

$$T_o = pT_P - T_S.$$

Performance Metrics for Parallel Systems

61

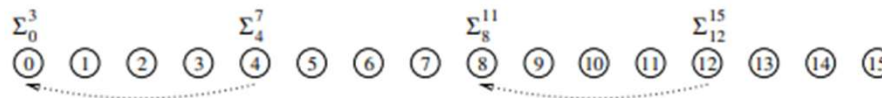
- **Performance Metrics #3:Speedup**
 - What is the benefit from parallelism
 - Speedup (S) is the ratio of the time taken to solve a problem on a single processor to the time required to solve the same problem on a parallel computer with p identical processing elements.
 - computing speedup, we always consider **the best sequential program** as the **baseline**



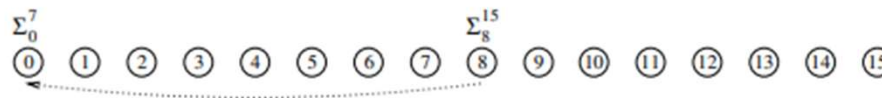
(a) Initial data distribution and the first communication step



(b) Second communication step



(c) Third communication step



(d) Fourth communication step



(e) Accumulation of the sum at processing element 0 after the final communication

- Speedup Example : Parallel bubble sort
 - serial time for bubblesort is 150 second
 - The parallel time for odd-even sort (efficient parallelization of bubble sort) is 40 seconds
 - The speedup would appear to be 3.75. (?)
 - But is this really a fair assessment of the system?
- What if serial quicksort only took 30 seconds? In this case, the speedup is $30/40 = 0.75$. This is a more realistic assessment of the system.

Performance Metrics for Parallel Systems

64

- **Performance Metrics #4:Efficiency**
 - **Efficiency** is a measure of the fraction of time for which a processing element is usefully employed; it is defined as the ratio of speedup to the number of processing elements.
 - **In an ideal parallel system, speedup is equal to p and efficiency is equal to one.**

$$E = \frac{S}{p}.$$

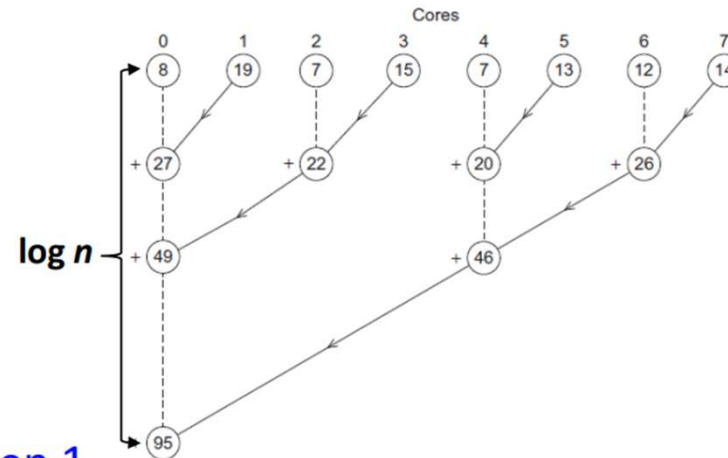
- **Performance Metrics #5: Cost**
 - Product of parallel execution time and number of Pes
 - The total amount of time by all PEs to solve the problem
 - Cost-optimal : parallel cost $\cong$ serial cost
 - ~ 0 overhead
 - $E = \Theta(1)$, since $E = T_s / p * T_p$

Cost: An Example

66

Adding n numbers on n PEs

- Serial performance: $T_s = \Theta(n)$
- Parallel performance: $T_p = \Theta(\log n)$
- Cost: $p T_p = \Theta(n \log n)$
- Optimal or not:
 - $E = n/n * \log n = \Theta(1/\log n)$
 - Not cost-optimal.
- Why not optimal
 - Waste of CPU cycles after step 1
 - Only core 0 is doing all the useful work in $\log N$ times



Amdahl's Law for Parallelism

67

- The enhanced fraction F is through parallelism, perfect parallelism with linear speedup
 - The speedup for F is N for N processors
 - Overall speedup

$$S(N) = \frac{T_s}{T_p} = \frac{T_s}{(1-F)*T_s + \frac{F*T_s}{N}} = \frac{1}{1-F + \frac{F}{N}}$$

M2 : Measures of Parallel Algorithms

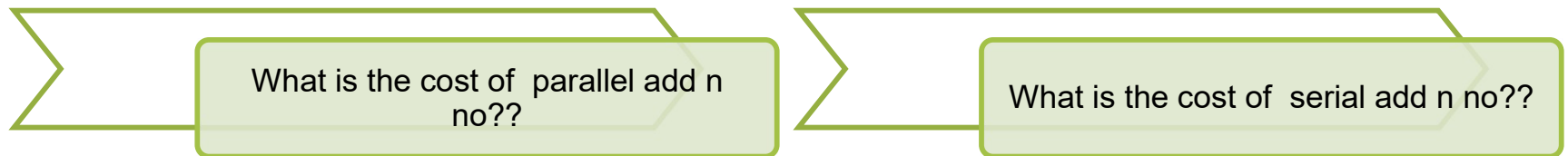
68

- Analytical Modeling of Parallel Programs:
 - Sources of Overhead in Parallel Programs
 - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Parallel Platforms: Models (SIMD, MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

What is Granularity

69

- The no and Size of task into the which problem size is decomposed determines granularity of the decomposition.
 - Smaller granularity
 - Larger Granularity
 - Scale down



The Effect of Granularity on Performance

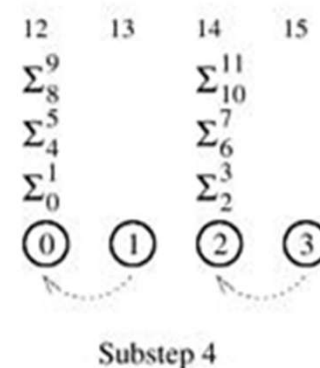
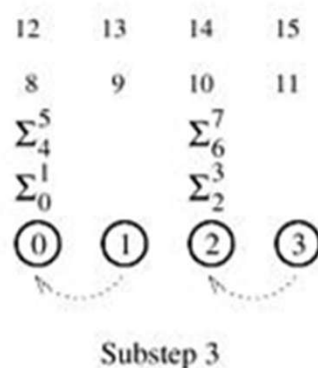
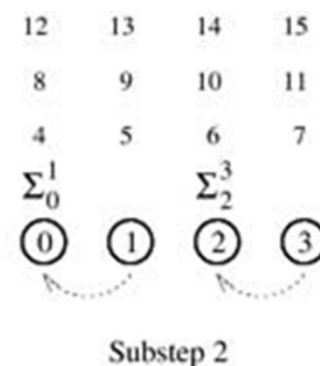
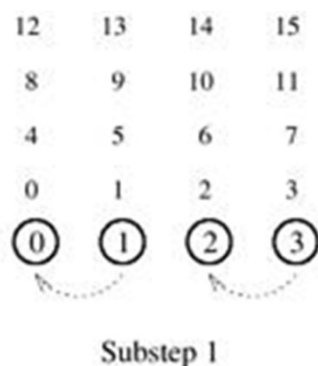
70

- **Scaling down :**
 - Often, using fewer processors improves performance of parallel systems.
 - Using fewer than the maximum possible number of processing elements to execute a parallel algorithm is scaling down.
 - number of processing elements decreases by a factor of n/p and the computation at each processing element increases by a factor of n/p .

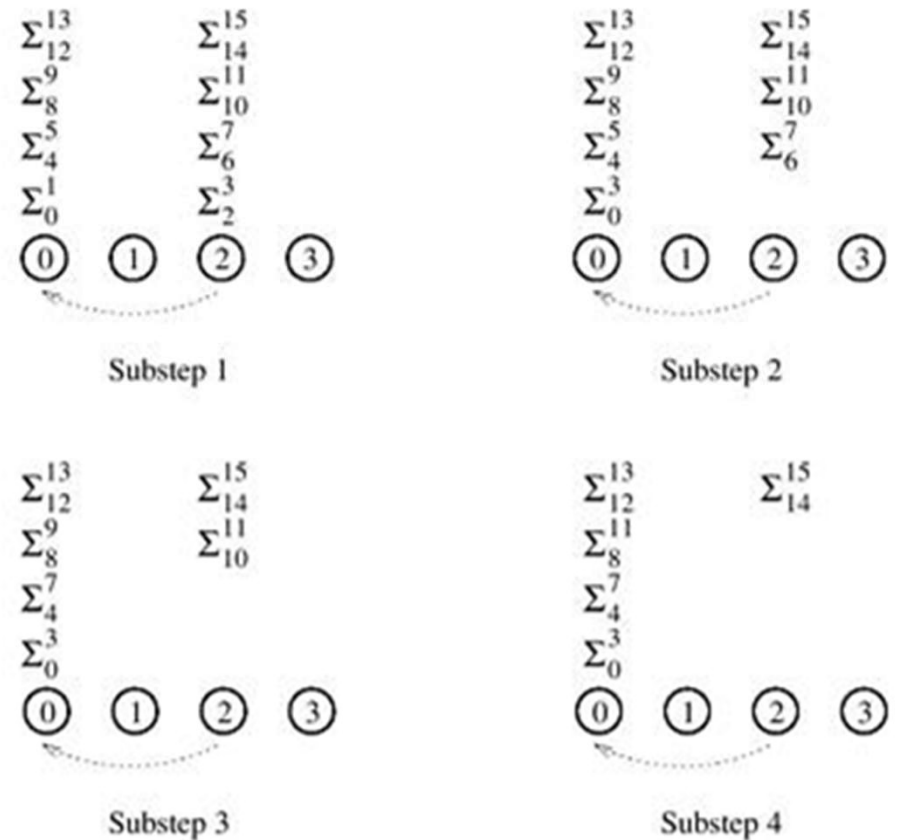
Adding n numbers on p processing elements

71

- Adding n numbers on p processing elements
 - for $n = 16$ and $p = 4$.
 - Virtual processing element i is simulated by the physical processing element labeled $i \bmod p$.

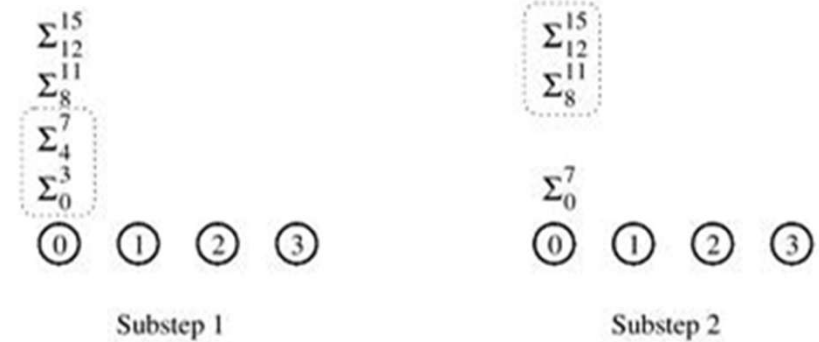


1. The first $\log p$ of the $\log n$ steps of the original algorithm are simulated in (n/p) $\log p$ steps on p processing element



(b) Four processors simulating the second communication step of 16 processors

1. n -processor machine
in time $\Theta(\log n)$.

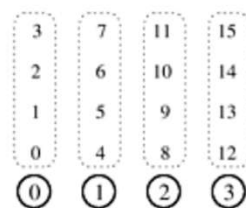


(c) Simulation of the third step in two substeps

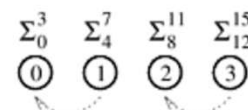


1. Adding n numbers cost-optimally:

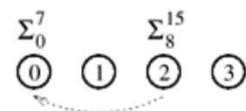
1. its cost is $\Theta(n + p \log p)$. As long as $n = \Omega(p \log p)$, the cost is $\Theta(n)$, which is the same as the serial runtime. Hence, this parallel system is cost-optimal.



(a)



(b)



(c)



(d)

Scaling Characteristics of Parallel Programs

75

- For a given problem size (i.e., the value of TS remains constant), as we increase the number of processing elements, T_o increases.
- The overall efficiency of the parallel program goes down. This is the case for all parallel programs.

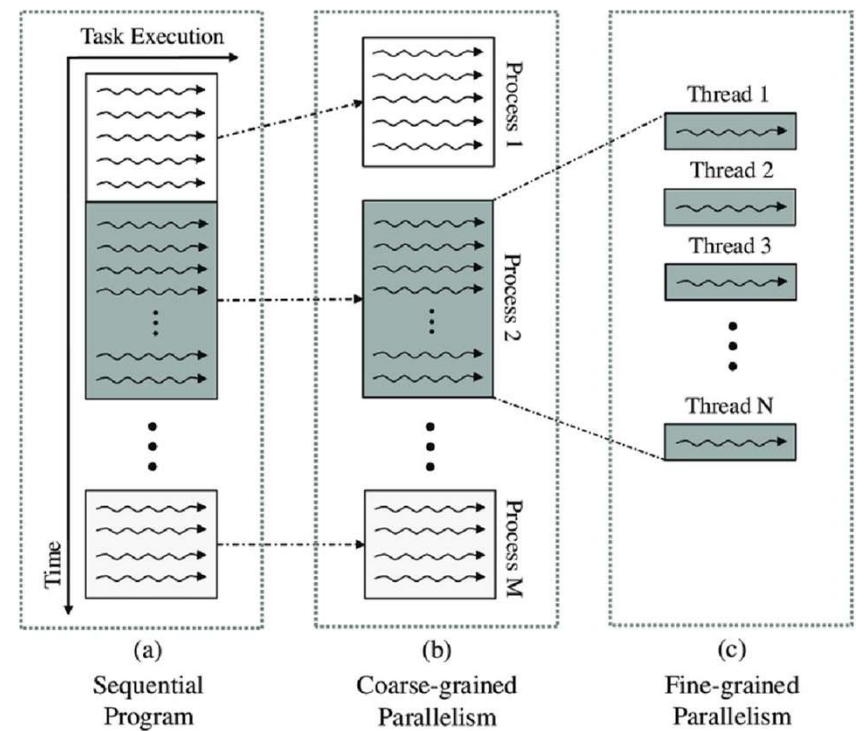
1. Granularity of Task Decomposition

- Fine-grained decomposition: large number of small tasks
- Coarse-grained decomposition: small number of large tasks
- Degree of Concurrency: # of tasks that can execute in parallel

- **Fine-grained parallelism**
 - – a program is broken down to a large number of small **tasks**.
 - – **Tasks** are assigned individually to many processors.
- The work is evenly distributed among the processors.
 - Increases the communication and synchronization overhead
- Fine-grained parallelism is best exploited in architectures which support fast communication
- **Shared memory** architecture are most suitable for fine-grained parallelism?
Why ?

- **Coarse-grained parallelism**

- a program is split into **large tasks**. a large amount of computation takes place in processors.
 - Load imbalance
- Certain tasks process the bulk of the data while others might be idle.
 - Low communication and synchronization overhead.



- Example :Consider a 10×10 image that needs to be processed, given that, processing of the 100 pixels is independent of each other. Assume there are
 - **Fine-grained parallelism:**
 - 100 processors that are responsible for processing the 10×10 image.
 - 100 processors can process the 10×10 image in 1 clock cycle
 - Each processor is working on 1 pixel of the image
 - **Medium-grained parallelism:**
 - 25 processors processing the 10×10 image.
 - The processing of the image will now take 4 clock cycles.
 - **Coarse-grained parallelism**
 - we reduce the processors to 2, processing will take 50 clock cycles

Relationship between levels of parallelism, grain size and degree of parallelism

<u>Levels</u>		Grain Size		Parallelism
Instruction level	--	Fine	--	Highest
Loop level	--	Fine	--	Moderate
Sub-routine level	--	Medium	--	Moderate
Program level	--	Coarse	--	Least

Finding the best grain size, depends on a number of factors and varies greatly from problem-to-problem.

- Building Granularity: Example
 - Consider the problem of adding n numbers on p processing elements such that $p < n$ and both n and p are powers of 2.